# NCollection ERP Platform — Master System Design & Execution Plan

**Version**: 5.0 **Date**: July 16, 2026 **Classification**: Internal — Enterprise Engineering Reference **Prepared By**: Architecture & Planning Team **Purpose**: The single authoritative technical reference and execution plan for the NCollection ERP Platform. Version 5.0 is a full revision of v4.0 incorporating the July 2026 planning review: rebalanced workloads, defense-in-depth license enforcement, OCA-first authentication, PITR backups, a dedicated provisioning runner, automated E2E testing, OCA dependency pinning, and two new deep-dive companion documents.

**Companion documents** (read together with this plan):

- ARCHITECTURE_DATA_PLATFORM.md — the database & distributed-data backbone: tenant registry, pooling topology, PITR, scaling stages, migration orchestration, and the platform-services (microservices) decomposition roadmap.
- ARCHITECTURE_SECURITY.md — the layered security architecture: threat model, tenant isolation guarantees, license enforcement design, authentication, secrets, encryption, and compliance (UAE PDPL).
- DELIVERABLE_2_TIMELINE_AND_TOOLING.md — timeline, sprints, GitHub strategy, CI/CD, releases, risks, budget.
- SPRINT_SCHEDULE.md — the resource-constrained sprint-by-sprint schedule: what runs in parallel vs series, per-developer utilization, and how idle gaps are filled by pulling later-phase work forward.
- PLANNING_REVIEW.md — the July 2026 review findings and the v4.0 → v5.0 task ID mapping.

---

## Table of Contents



---

## 1. Project Overview & Current State

### 1.1 What NCollection ERP Is

NCollection ERP is a **commercial SaaS ERP Platform** built on top of **Odoo 19 Community Edition**, targeting small-to-medium businesses (5–100 employees) across the **UAE and GCC region**.

**Critical distinction**: The project is NOT an Odoo customization. Odoo is the **ERP engine**. The real product is the **SaaS platform layer** around Odoo — tenant management, subscription licensing, provisioning, white-label branding, and UAE localization. This platform will eventually serve **thousands of companies across the GCC**, which means every design decision must assume: multiple hostile-by-default tenants sharing infrastructure, strict data isolation, and an additional platform layer that must never leak into (or be leaked by) the ERP layer beneath it.

### 1.2 Completed Milestones (DO NOT REDESIGN)

The following milestones are **complete and stable**. They must not be recreated, redesigned, or refactored unless explicitly requested:

| Milestone | Status | What Was Delivered |
|---|---|---|
| **Docker Infrastructure** | ✅ Complete | `docker-compose.yml` with PostgreSQL 16 + Odoo 19 containers, volume mounts, custom_addons directory |
| **PostgreSQL** | ✅ Complete | PostgreSQL 16 running in Docker, `odoo` role configured, persistent volume |
| **White Label / Branding** | ✅ Complete | `ncollection_branding` module — custom logo, login page, favicon, SCSS theme colors, browser title override |
| **SaaS Foundation** | ✅ Complete | `ncollection_subscription` module — Tenant, Subscription, Plan, Provisioning Job models with full ORM, chatter integration, computed fields |
| **Organizations** | ✅ Complete | `ncollection.tenant` model with UUID, database status tracking, onboarding stages, contact info, plan linking |
| **Subscription Plans** | ✅ Complete | `ncollection.subscription.plan` model with tiered pricing (monthly/yearly), user limits, currency support |
| **Provisioning Queue** | ✅ Complete | `ncollection.provisioning.job` model with status tracking (queued/running/done/failed), logging |
| **Financial Reporting** | ✅ Complete | OCA `account_financial_report` installed — General Ledger, Trial Balance, Journal Ledger, VAT Report, Open Items, Aged Partner Balance |
| **MIS Builder** | ✅ Complete | OCA `mis_builder` installed with custom `ncollection_mis_templates` (Balance Sheet, P&L) |
| **Fresh Origin Demo** | ✅ Complete | Demo company with sample data across CRM, Sales, Purchase, Inventory, HR, Projects |
| **Dashboard Redesign** | ✅ Complete | `ncollection.subscription.dashboard` transient model with KPI computations (total tenants, MRR, expiring subscriptions) |
| **CI/CD Foundation** | ✅ Complete | GitHub Actions pipeline with flake8 linting on PRs to `develop` and `main` |

### 1.3 Current Project State — What Exists in Code

**Repository**: `NCollection-Sys/ncollection-erp` (Private GitHub)

```
ncollection-erp/
├── custom_addons/
│   ├── ncollection_branding/        ✅ Implemented (partial — pending items remain)
│   ├── ncollection_subscription/    ✅ Implemented (models + views + demo data)
│   ├── ncollection_core/            🔲 Skeleton only (manifest + empty init)
│   └── ncollection_saas/            🔲 Skeleton only (manifest + empty init)
├── docs/                            ✅ PRD, Roadmap, System Design, Architecture deep-dives
├── scripts/github_issue_sync.py     ✅ Task → GitHub Issue importer
├── .github/workflows/ci.yml         ✅ flake8 on PRs
└── docker-compose.yml               ✅ PostgreSQL 16 + Odoo 19
```

### 1.4 Current Sprint: Customer Workspace

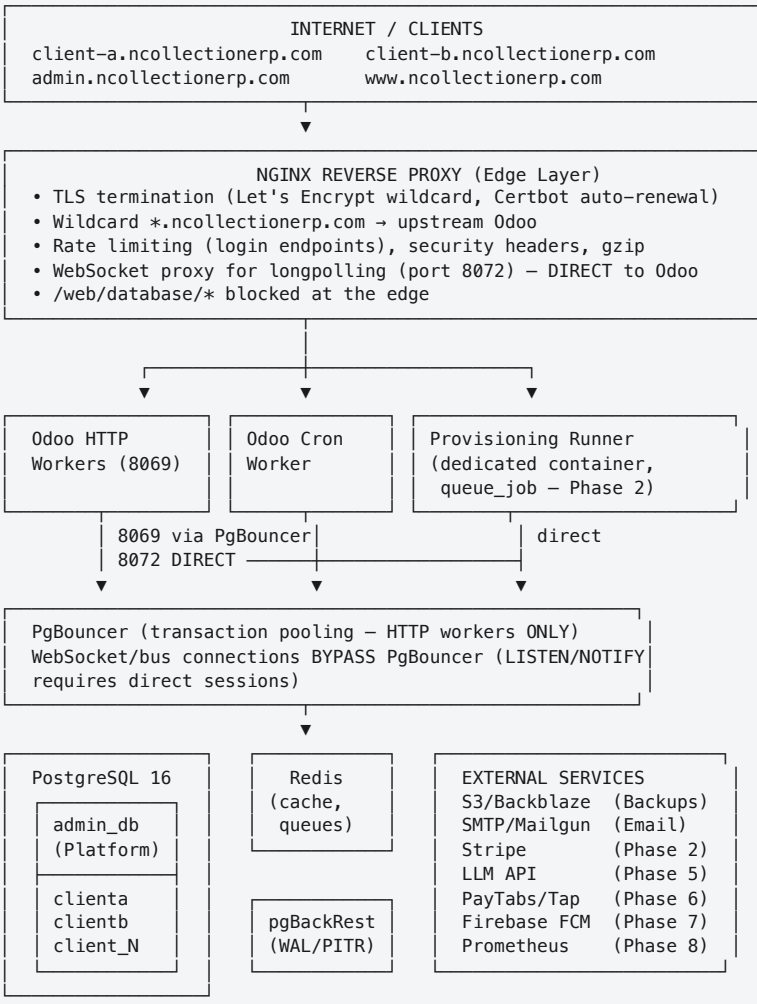
The team must focus **exclusively** on the **Customer Workspace** phase (Phase 1 below). Everything else is lower priority and must not be started until Customer Workspace is complete, tested, and stable.

## 2. System Architecture

> This section is the executive summary. The full backend/database design — connection pooling topology, PITR, tenant registry, scaling stages, service decomposition — lives in **ARCHITECTURE_DATA_PLATFORM.md**. The full security design lives in **ARCHITECTURE_SECURITY.md**.

### 2.1 Architecture Overview

The NCollection ERP Platform follows a **database-per-tenant** multi-tenant SaaS architecture, where a single Odoo 19 deployment serves multiple isolated PostgreSQL databases, each belonging to a different subscribing company.




### 2.2 Database-per-Tenant Isolation Strategy

**Chosen Model**: Database-per-Tenant (same model as Odoo.com SaaS)

**Reason**: This is the only model that provides complete data isolation, independent backups, independent migrations, and zero risk of cross-tenant data leakage. For a GCC-targeted commercial SaaS platform, data sovereignty and isolation are non-negotiable requirements.

**Benefits**:

- Complete data isolation between tenants — impossible to accidentally query another tenant's data
- Independent backup and restore per tenant without affecting others

- Independent Odoo module upgrades per tenant (can roll out changes gradually)
- Compliance-friendly — each tenant's data can be stored/deleted independently (UAE PDPL)
- Follows the proven Odoo.com architecture

**Risks & Mitigations**:

- PostgreSQL connection count grows linearly with tenant count → PgBouncer transaction pooling (see ARCHITECTURE_DATA_PLATFORM.md §4)
- Cron jobs run per-database → dedicated cron worker + queue_job runner
- Schema migrations must be applied to all tenant databases → migration orchestrator with canary tenants (see ARCHITECTURE_DATA_PLATFORM.md §7)

**Alternatives Considered**:

| Alternative | Why Rejected |
|---|---|
| Multi-company (single DB) | No true isolation; risk of data leakage; cannot do independent backups; one bad migration affects everyone |
| Schema-per-tenant | Odoo doesn't support this natively; would require deep core modifications violating Rule 1 |
| Row-level security | Same as multi-company; insufficient for enterprise SaaS compliance requirements |

## 2.3 Subdomain Routing Mechanism

**How It Works**:

```
1. DNS: *.ncollectionerp.com → VPS IP (wildcard A record)
2. Nginx: Receives request for clienta.ncollectionerp.com
3. Nginx: Proxies to Odoo with Host header preserved
4. Odoo: db_filter = ^%d$ extracts "clienta" from the hostname
5. Odoo: Routes the request to database "clienta"
6. Odoo: Session cookie is scoped to that database only
```

> [!IMPORTANT] **`%d` vs `%h` — a correction from v4.0**: In Odoo's `db_filter`, `%h` is replaced by the **full hostname** (`clienta.ncollectionerp.com`) while `%d` is replaced by the **first subdomain component** (`clienta`). Since tenant databases are named after the subdomain (`clienta`), the correct filter is **`db_filter = ^%d$`**. Using `^%h$` (as v4.0 stated) would require databases literally named `clienta.ncollectionerp.com` and would break routing.

**Odoo Configuration** (`config/odoo.conf`):

```
[options]
; --- Multi-Tenant Routing ---
db_host = db
db_port = 5432
db_user = odoo
db_password = ${DB_PASSWORD}
db_name = False                          ; Allow connections to any database
db_filter = ^%d$                         ; Route by SUBDOMAIN (CRITICAL — see note above)
list_db = False                          ; SECURITY: Hide database selector completely

; --- Performance (4 vCPU / 8 GB RAM baseline) ---
workers = 4                              ; (2 × CPU_cores) + 1 guideline, tuned in P3-T03
max_cron_threads = 1                     ; Dedicated cron worker
limit_memory_hard = 2684354560           ; 2.5 GB per worker
limit_memory_soft = 2147483648           ; 2.0 GB per worker
limit_time_cpu = 600
limit_time_real = 1200
limit_time_real_cron = 3600

; --- Security ---
admin_passwd = ${ADMIN_MASTER_PASSWORD}  ; Strong, unique; DB management API disabled in prod
proxy_mode = True                        ; Trust Nginx X-Forwarded headers

; --- Paths ---
addons_path = /mnt/extra-addons,/mnt/oca-addons,/usr/lib/python3/dist-packages/odoo/addons

; --- Logging ---
log_level = info
log_handler = :INFO,werkzeug:WARNING
```

## 2.4 Isolation Guarantees

| Layer | Mechanism | Verification |
|---|---|---|
| **Database** | Each tenant has its own PostgreSQL database — no shared tables | `SELECT datname FROM pg_database` shows separate DBs |
| **Application** | `db_filter = ^%d$` ensures each HTTP request only accesses the matched database | Automated E2E test (P1-T20): `clienta` subdomain can never read `clientb` data |
| **Session** | Odoo sessions are database-scoped — cookies are tied to the specific DB | E2E test: login on `clienta`, visit `clientb` → must see login page |
| **License** | Menu hiding (UI) **plus** ORM/RPC enforcement (security) — see ARCHITECTURE_SECURITY.md §4 | Automated test: RPC call to an unlicensed model is denied |
| **Filestore** | Attachments stored per database directory — physically separated | Directory listing shows separate folders per tenant |
| **Network** | Nginx enforces subdomain routing; `/web/database/*` blocked at the edge | Port scan + URL probing in the P1-T21 security audit |

## 2.5 Per-Database Module Installation Matrix

> **New in v5.0.** The v4.0 plan never stated which addon runs where — a recurring source of confusion. This matrix is authoritative:

| Module | admin_db (Platform) | Tenant DBs | Notes |
|---|---|---|---|
| `ncollection_subscription` | ✅ | ❌ | Tenants, plans, subscriptions, provisioning jobs — platform staff only |
| `ncollection_saas` | ✅ | ❌ | Provisioning automation, billing, domains — platform staff only |
| `ncollection_core` | ❌ | ✅ | Roles, workspace config, module visibility & license enforcement |
| `ncollection_branding` | ✅ | ✅ | White-label everywhere |
| `ncollection_uae` | ❌ | ✅ | VAT, CoA, currency — per tenant (plan-dependent) |
| `ncollection_ai` (Phase 5) | ❌ | ✅ | Gateway config is platform-side; widget/context run per tenant |

| | | | |
|---|---|---|---|
| OCA financial modules | ❌ | ✅ | Installed by provisioning when the plan includes accounting |
| Odoo business modules (crm, sale, …) | minimal | ✅ | Tenant DBs get exactly the plan's licensed set |

The **provisioning engine (P2-T01/T02) is the only writer** of tenant module sets, and the **workspace config sync (P2-T03)** is the only channel that updates them afterwards.

### 2.6 Security Architecture (Summary)

Full detail, threat model, and rationale: **ARCHITECTURE_SECURITY.md**.

| Layer | Mechanism | Delivered By |
|---|---|---|
| **Edge** | TLS everywhere, HSTS, rate limiting, security headers, `/web/database/*` blocked | P1-T03 |
| **Application** | OCA-based brute-force protection & session timeout, auth audit log, hardened cookies | P1-T19 |
| **Authorization** | 8 predefined roles, Apps/Settings stripping, Owner-only workspace settings | P1-T08/T11/T12 |
| **License enforcement** | Menu hiding + ORM `ir.rule` /ACL enforcement + RPC guard (defense in depth) | P1-T09/T10 |
| **Database** | Not exposed publicly; PgBouncer topology; least-privilege roles | P2-T08/T09 |
| **Data** | PITR (pgBackRest), encrypted off-site backups, restore drills | P2-T04/T05 |
| **Operations** | UFW, SSH keys only, fail2ban, image/dependency scanning in CI, secrets in `.env` →Docker secrets | P2-T08, P1-T05 |
| **Assurance** | Automated cross-tenant E2E tests, pre-launch security assessment, audit trail | P1-T20/T21, P3-T12, P8-T05 |

---

## 3. Two-Layer Architecture Philosophy

> **Rule 3**: Platform Layer and ERP Layer are completely different responsibilities. Never mix them.

This is the single most important architectural principle of the NCollection ERP Platform. Every developer, every code review, and every design decision must respect this boundary.

### 3.1 Platform Layer (NCollection SaaS)

**Responsibility**: The business of selling and operating ERP as a service.

| Component | Description | Module |
|---|---|---|
| Organizations | Tenant companies, UUIDs, onboarding stages | `ncollection_subscription` |
| Plans | Subscription tiers, pricing, module licensing | `ncollection_subscription` |
| Subscriptions | Tenant↔Plan linking, billing cycle, status lifecycle | `ncollection_subscription` |
| Provisioning | Database creation, module installation, admin user setup | `ncollection_saas` (dedicated runner) |
| Billing & Payments | Invoice generation + Stripe collection for SaaS subscriptions | `ncollection_saas` (Phase 2) |
| Domains | Subdomain assignment, SSL management | `ncollection_saas` (Phase 2) |
| Licensing | Module visibility + enforcement per subscription | `ncollection_core` (Phase 1) |
| Monitoring | Platform health, tenant DB sizes, request metrics | Phase 2 (lightweight) → Phase 8 (full) |

**Accessed by**: NCollection platform administrators only (never by tenant end-users).

### 3.2 ERP Layer (Odoo 19 Community)

**Responsibility**: The actual business operations for each subscribing company.

| Component | Odoo Module | Customization Approach |
|---|---|---|
| CRM | `crm` | XML view inheritance, Python `_inherit` |
| Sales | `sale` | Workflow enhancements via custom addon |
| Purchase | `purchase` | Approval workflows via custom addon |
| Inventory | `stock` | Barcode endpoints via custom addon |
| Accounting | `account` + OCA modules | UAE localization via `ncollection_uae` |
| HR | `hr` | Attendance, leave management via OCA |
| Projects | `project` | Standard usage |

**Accessed by**: Tenant end-users (employees of subscribing companies).

### 3.3 How the Layers Interact

```
Platform Layer (admin.ncollectionerp.com — admin_db)
    │
    │ 1. Customer self-registers via public checkout (or admin creates Organization)
    │ 2. Subscription Plan assigned (trial or paid)
    │ 3. Provisioning runner creates the tenant DB (isolated container)
    │ 4. Runner installs exactly the licensed modules
    │ 5. Runner creates the tenant admin user + writes workspace config
    │ 6. Runner applies branding defaults, registers domain + SSL
    │ 7. Plan changes propagate later via the config sync channel (P2-T03)
    ▼
ERP Layer (clienta.ncollectionerp.com — clienta DB)
    │
    │ Tenant users log in and see ONLY their licensed modules
    │ License enforcement blocks unlicensed models even via RPC
    │ Zero visibility into the Platform Layer
    ▼
Tenant's End Customers (Portal — Phase 6)
    │
    │ Portal users see their own invoices, orders, tickets — nothing else
```

---

## 4. Existing Modules & OCA Integrations

### 4.1 Custom NCollection Modules (Existing)

| Module | Status | Description | Key Models |
|---|---|---|---|
| `ncollection_branding` | ✅ Partial | White-label: logo, favicon, title, SCSS colors | — (template overrides) |
| `ncollection_subscription` | ✅ Core | SaaS foundation: tenants, plans, subscriptions, provisioning, dashboard | `ncollection.tenant` , `ncollection.subscription` , `ncollection.subscription.plan` , `ncollection.provisioning.job` , `ncollection.subscription.dashboard` |
| `ncollection_core` | 🔲 Skeleton | Will hold: roles, access rights, module visibility + license enforcement, workspace config | — |
| `ncollection_saas` | 🔲 Skeleton | Will hold: provisioning automation, billing, payments, domain management | — |

## 4.2 Planned Custom Modules

| Module | Phase | Description |
|---|---|---|
| `ncollection_uae` | 3 | UAE localization: VAT, CoA, AED, Arabic, invoice templates |
| `ncollection_ai` | 5 | AI platform: LLM gateway, context engine, chat widget |
| `ncollection_portal` | 6 | Portal redesign and customer-facing ticketing |
| `ncollection_api` | 8 | Public REST API with OAuth2 |
| `ncollection_marketplace` | 9 (deferred) | Integration marketplace |

## 4.3 OCA Modules (Installed)

| Module | OCA Repository | Branch | Status |
|---|---|---|---|
| `account_financial_report` | `OCA/account-financial-reporting` | 19.0 | ✅ Installed |
| `mis_builder` | `OCA/mis-builder` | 19.0 | ✅ Installed |

> [!WARNING] These were initially cloned manually. **P1-T04** converts all OCA dependencies to pinned, reproducible management via `git-aggregator` ( `repos.yml` with commit hashes) so every developer, CI, and production run identical code.

## 4.4 OCA Modules to Evaluate Before Custom Development

> **Rule 2**: Always search OCA before suggesting new development. Never reinvent mature OCA modules.

| Need | OCA Repository to Check | Task Affected |
|---|---|---|
| Brute-force login protection | `OCA/server-auth` → `auth_brute_force` | P1-T19 |
| Session timeout | `OCA/server-auth` → `auth_session_timeout` | P1-T19 |
| 2FA / TOTP | `OCA/server-auth` | P10-T04 |
| Queue / async jobs | `OCA/queue` → `queue_job` | P2-T01 |
| Audit Trail | `OCA/server-tools` → `auditlog` | P8-T05 |
| REST API | `OCA/rest-framework` → `base_rest` / FastAPI addon | P8-T01 |
| Webhooks | `OCA/server-tools` | P8-T03 |
| Currency rates | `OCA/currency` | P3-T06 |
| Helpdesk/Ticketing | `OCA/helpdesk` | P6-T03 |
| Bank reconciliation | `OCA/account-reconcile` | P10-T06 |
| PDF/report tooling | `OCA/reporting-engine` | P3-T09 |

**Process**: Before starting any task, the assigned developer must:

1. Search the relevant OCA repository for an Odoo 19-compatible module
2. If found: evaluate fit, install in dev environment, test compatibility
3. If suitable: pin it in `repos.yml` and document the integration
4. If not suitable (or no 19.0 port exists): document WHY, then build custom
5. Log the decision in the GitHub Issue for the task

---

## 5. Team Structure & Personas

### 5.1 Human Development Team

#### [DEV-1] Backend & Infrastructure Lead

**Core Skills**: Python 3.12+, PostgreSQL 16, Docker/Docker Compose, Linux (Ubuntu), Nginx, CI/CD, REST API design, shell scripting, security hardening.

**Responsibilities**:

- Owns the entire infrastructure layer: Docker, Nginx, PostgreSQL, CI/CD, server provisioning
- Owns the data platform: pooling topology, PITR/backups, replication, migration orchestration (ARCHITECTURE_DATA_PLATFORM.md)
- Designs and operates SaaS automation: provisioning runner, database creation, domain/SSL management
- Builds the DB routing engine and the authentication hardening layer
- Implements API layers (REST, OAuth2, mobile API optimization)
- Owns security operations, performance tuning, and monitoring

**Module Ownership**: Infrastructure configs, `ncollection_saas` (provisioning/ops), `ncollection_api`, monitoring.

#### [DEV-2] Odoo & Business Logic Specialist

**Core Skills**: Odoo ORM, XML views (form/list/kanban/search), QWeb reports, Access Rights ( `ir.model.access` , `ir.rule` ), Odoo Accounting, Workflows, OCA module integration.

**Responsibilities**:

- Owns all business logic: model definitions, computed fields, constraints, state machines
- Implements module visibility engine AND ORM-level license enforcement
- Defines `res.groups` , access rights, and record rules for tenant role isolation
- Handles ERP enhancements: CRM, Sales, Purchase workflows
- Owns UAE localization: VAT, Chart of Accounts, currency
- Implements billing engine, payment collection (Odoo payment providers), KPI logic, anomaly detection

**Module Ownership**: `ncollection_subscription` (business logic), `ncollection_core` (roles/access/licensing), `ncollection_uae` .

#### [DEV-3] Frontend & Integration Specialist

**Core Skills**: OWL framework, JavaScript (ES6+), QWeb templates, SCSS/CSS, responsive design, mobile development (React Native / Flutter), UI/UX design, Playwright E2E testing.

**Responsibilities**:

- Owns the branding system: `ncollection_branding` completion (logos, login page, About dialog)
- Builds dynamic per-tenant branding (CSS variable injection) and the Owner's workspace settings UI
- Develops all dashboard UIs: Customer Dashboard, CEO Dashboard, Department Dashboards (OWL + charting)
- Owns the Playwright E2E test framework and UI test journeys
- Designs and implements the self-service checkout flow and customer portal redesign
- Creates the AI Assistant chat widget (OWL) and develops the mobile application

**Module Ownership**: `ncollection_branding` , all UI components, dashboard widgets, portal templates, E2E suite, mobile app.

### 5.2 AI Engineering Team

| AI Agent | Role | Responsibility |
|---|---|---|
| **ChatGPT** | Chief Solution Architect | High-level architecture decisions, business logic design, OCA module evaluation |
| **Claude** | Implementation Engineer | Code generation, pair programming, debugging, code review assistance |

| | | |
|---|---|---|
| **Gemini** | Architecture Reviewer & Planning Assistant | Architecture review, planning documents, dependency analysis, risk assessment |

**AI Collaboration Rules**:

1. AI agents understand the current milestone before generating code
2. AI agents verify architecture alignment with the two-layer philosophy
3. AI agents avoid regressions and maintain Odoo 19 Community compatibility
4. AI agents build features incrementally — never generate entire modules in one shot
5. AI agents NEVER jump to future phases unless the current phase is complete
6. AI agents follow **Rule 6 (Odoo 19 conventions)** — `<list>` not `<tree>`, no deprecated `attrs=`, no legacy widget syntax

---

## 6. Development Rules & Principles

These rules are **mandatory and non-negotiable**. Every developer, every code review, and every AI-generated code must comply.

### Rule 1: Never Modify Odoo Core

**Reason**: Modifying Odoo core files creates unsustainable technical debt. Any change will be overwritten during Odoo version upgrades.

**Approved Extension Methods**:

| Method | When to Use | Example |
|---|---|---|
| **Custom Addons** | Always the primary approach | `ncollection_branding`, `ncollection_subscription` |
| **Python `_inherit`** | Extending existing models | `class ResCompany(models.Model): _inherit = 'res.company'` |
| **XML View Inheritance** | Modifying existing views | `<template inherit_id="web.login">` |
| **OWL Component Patching** | Modifying frontend components | `patch(WebClient.prototype, { ... })` |
| **SCSS Overrides** | Changing visual styles | Custom SCSS in asset bundles |
| **Controller Override** | Modifying HTTP routes | `class CustomHome(Home): @route('/web/login')` |

### Rule 2: OCA First

The OCA maintains 1000+ battle-tested modules. Before ANY new development → search OCA → evaluate → decide → document. This now explicitly includes **authentication** (`auth_brute_force`, `auth_session_timeout`), **async jobs** (`queue_job`), and **audit** (`auditlog`).

### Rule 3: Two-Layer Separation

See Section 3. Platform Layer and ERP Layer must never mix code, models, or views. The module installation matrix (§2.5) is the enforcement reference.

### Rule 4: Upgrade Compatibility

- No fragile overrides that depend on internal Odoo implementation details
- Use public API methods; document every `_inherit` override with the reason and the Odoo method being overridden
- Pin OCA module versions via `git-aggregator` `repos.yml` (P1-T04)

### Rule 5: Enterprise SaaS Mindset

**Checklist for every feature**:

- [ ] Does this work correctly across 100+ tenant databases?
- [ ] Does this maintain tenant data isolation?
- [ ] Does this impact Odoo startup time or memory usage?

- [ ] Can this be configured per-tenant without code changes?
- [ ] Does this preserve upgrade compatibility?

### Rule 6: Odoo 19 View & Code Conventions (NEW in v5.0)

**Reason**: Odoo 19 deprecated several patterns; using them fails module installation or breaks silently.

- Views: use `<list>` — `<tree>` is deprecated/removed in Odoo 19
- View attributes: use direct `invisible="condition"` / `readonly="condition"` expressions — the legacy `attrs="{...}"` dict is removed
- Frontend: OWL 2 components only; no legacy `odoo.define` AMD modules
- Backend URLs live under `/odoo/...` in Odoo 19 — do not hardcode legacy `/web#` fragments
- Python: 3.12+, type annotations on new code, no `print()` (use `_logger`)
- Every developer and AI agent must be prompted with these conventions before generating view XML

### Rule 7: Security Is a Feature, Not a Phase (NEW in v5.0)

**Reason**: The platform hosts many companies' financial data on shared infrastructure with a custom layer on top of Odoo. A single isolation or licensing bypass is an existential business risk.

- UI-level hiding is never sufficient — every restriction must also exist at the ORM/RPC layer
- Every phase gate includes the cross-tenant isolation test suite (established in P1-T20/T21)
- Security-sensitive PRs (auth, access rights, provisioning, payments) require review against ARCHITECTURE_SECURITY.md checklists
- Secrets never enter git; dependency and image scanning run in CI

---

## 7. Collaboration Workflow

### 7.1 Git Branching Strategy

```
main (production-ready, always deployable)
  │
  └── develop (integration branch — all PRs merge here)
        │
        ├── feature/P1-T02-odoo-conf-multitenant      (DEV-1)
        ├── feature/P1-T09-module-visibility          (DEV-2)
        ├── feature/P1-T13-branding-completion        (DEV-3)
        ├── hotfix/login-csrf-fix                     (any DEV)
        └── ...
```

**Rules**:

1. `main` is always deployable. Only `develop` merges into `main` via release PRs.
2. All feature branches merge into `develop` via reviewed PRs.
3. Feature branches follow naming: `feature/P{phase}-T{task_id}-{short-description}`.
4. Hotfix branches: `hotfix/{description}` — merge into both `main` and `develop`.

### 7.2 Commit Message Convention

```
[P1-T02] feat: Multi-tenant odoo.conf with subdomain db_filter

- db_filter=^%d$, list_db=False, proxy_mode=True
- .env-based secrets, dev/prod compose overrides
- Health checks for db and odoo services

Closes #42
```

Format: `[P{phase}-T{id}] {type}: {description}` — Types: `feat`, `fix`, `refactor`, `docs`, `test`, `chore`, `perf`, `security`.

---

## 8. Phase Execution Order & Gates

> **New in v5.0.** Phase numbers are stable identifiers (labels, milestones) — but they are NOT the strict execution order.

| Order | Phase | Priority | Overlap Notes |
|---|---|---|---|
| 1 | Phase 1 — Customer Workspace | **P0** | THE ONLY FOCUS right now |
| 2 | Phase 2 — SaaS Automation | P1 | DEV-2/DEV-3 begin Phase 3 tasks during Phase 2 (DEV-1 is the Phase 2 bottleneck by design) |
| 3 | Phase 3 — ERP + UAE | P1 | Ends with the **go-live gate (P3-T13)** → FIRST PRODUCTION DEPLOYMENT |
| 4 | Phase 4 — Dashboards | P2 | Lightweight; also feeds the AI context engine |
| 5 | Phase 6 — Customer Portal | P2 | **Pulled ahead of AI**: portal + regional payments drive revenue/retention |
| 6 | Phase 5 — AI Platform | P3 | Experimental; starts with a mandatory design spike (P5-T01) |
| 7 | Phase 7 — Mobile | P3 | New stack; framework decision documented in P7-T05 |
| 8 | Phase 8 — Platform Services | P3 | REST API + full observability |
| 9 | Phase 10 — Enterprise Readiness | P3 | HA, scaling, compliance — **before** Marketplace |
| 10 | Phase 9 — Marketplace | **DEFERRED** | Build a third-party ecosystem only after the core is enterprise-grade and there is proven tenant demand |

**Phase gates**: Phases 1, 2, and 3 each end with an explicit integration/security gate task (P1-T21, P2-T18, P3-T13). No phase is "done" until its gate passes and the regression checklist is green. Later phases re-run the cumulative regression + isolation suite.

---

## Phase 1: Customer Workspace — CURRENT SPRINT

**Priority**: P0 — IMMEDIATE — **THE ONLY FOCUS Objective**: Build the foundational SaaS experience where subscribers land directly in their isolated ERP workspace, see only their licensed modules — enforced at the UI **and** ORM layer — and experience NCollection branding with zero Odoo references.

> **IMPORTANT**: This phase builds ON TOP of the already-completed SaaS foundation. The tenant, subscription, and plan models ALREADY EXIST. This phase focuses on the CUSTOMER-FACING experience — what the tenant's employees see when they log into their workspace.

**Acceptance Criteria (Definition of Done)**:

- [ ] Users can log in via `company.ncollectionerp.com`
- [ ] Users land directly in their ERP workspace (NOT a SaaS admin panel)
- [ ] Users see ONLY their licensed modules — and unlicensed models are **inaccessible via direct URL and RPC**, not merely hidden
- [ ] Users CANNOT access any SaaS administration screens
- [ ] Users CANNOT see or access any other tenant's data (verified by automated E2E tests)
- [ ] Role-based access control works for all 8 roles; Owner manages users via a safe settings surface
- [ ] Tenant can customize their own logo, colors, and company information
- [ ] All visible branding says "NCollection ERP" — zero Odoo references anywhere (UI + emails)
- [ ] Customer Dashboard shows business KPIs relevant to the tenant and the viewer's role
- [ ] Playwright E2E suite covers login, routing, isolation, and visibility — running in CI

## Phase 1 Tasks

| ID | Task Name | Description | Assigned | Dependencies | Est. Days |
|---|---|---|---|---|---|
| **P1-T01** | Addon Skeleton & Test Scaffolding | Complete the EXISTING skeleton modules `ncollection_core` and `ncollection_saas` so every developer can start addon work on day 1. Scope: (1) proper `__init__.py` imports and `__manifest__.py` metadata with correct dependencies, (2) placeholder directories: `models/`, `views/`, `security/`, `data/`, `controllers/`, `wizards/`, `tests/`, (3) a smoke `TransactionCase` test in each module so the CI test runner has something to execute, (4) verify `odoo-bin -i ncollection_core,ncollection_saas --stop-after-init` exits cleanly.<br>**Reason**: nothing else in this phase can start without importable module structures — this task has NO dependency on infrastructure work and must be finished on Day 1.<br>**Acceptance**: both modules install without errors; smoke tests run green locally. | `[DEV-2]` | None | 1 |
| **P1-T02** | Multi-Tenant Odoo Configuration & Secrets | Create `config/odoo.conf` with all multi-tenant settings: `db_filter=^%d$` (NOTE: `%d` extracts the subdomain — `%h` is the FULL hostname and would require databases named `clienta.ncollectionerp.com`; v4.0 had this wrong), `list_db=False`, `proxy_mode=True`, workers and memory/time limits per §2.3. Create `.env` for secrets (DB password, admin master password), add it to `.gitignore`, and provide `.env.example`. Create `docker-compose.dev.yml` override (pgadmin, single worker, debug logging, mounted source) and a `docker-compose.prod.yml` stub. Add Docker health checks for db and odoo services.<br>**Reason**: the existing compose file works for development but has hardcoded credentials and no multi-tenant routing configuration.<br>**Acceptance**: `docker compose up` boots Odoo with the new config; the database selector is hidden; all secrets come from `.env`. | `[DEV-1]` | None | 2 |
| **P1-T03** | Nginx Reverse Proxy & TLS | Add an `nginx` service to the compose stack: wildcard server block for `*.ncollectionerp.com`, TLS termination via Certbot (wildcard certificate through the DNS-01 challenge), HTTP→HTTPS redirect, WebSocket proxy for longpolling on port 8072 (routed DIRECT to Odoo — never through a DB pooler), rate limiting on `/web/login` (`limit_req_zone`), gzip, and security headers (HSTS, X-Frame-Options, CSP). Block `/web/database/*` at the edge. Include a dev config for `*.localhost` domains without TLS.<br>**Reason**: Nginx is the routing and security front door for the whole multi-tenant platform.<br>**Risks**: wildcard cert automation requires DNS provider API access — document the manual fallback.<br>**Acceptance**: `https://anything.ncollectionerp.com` reaches Odoo with the correct Host header; security headers present; `/web/database/manager` returns 403 from the edge. | `[DEV-1]` | P1-T02 | 3 |
| **P1-T04** | OCA Dependency Management | Replace manual OCA clones/symlinks with reproducible dependency management. (1) Introduce `git-aggregator` with a `repos.yml` pinning `OCA/account-financial-reporting` and `OCA/mis-builder` (and all future OCA repos) to exact 19.0 commit hashes, (2) aggregate into `/mnt/oca-addons` mounted alongside `custom_addons`, (3) wire into the Docker build and CI so every environment resolves identical code, (4) document the update workflow (bump hash → PR → CI).<br>**Reason**: manual symlinks guarantee works-on-my-machine failures across a 3-person remote team, CI, and production.<br>**Acceptance**: a fresh clone + `gitaggregate -c repos.yml` + `docker compose up` reproduces the exact current environment on any machine. | `[DEV-1]` | None | 2 |

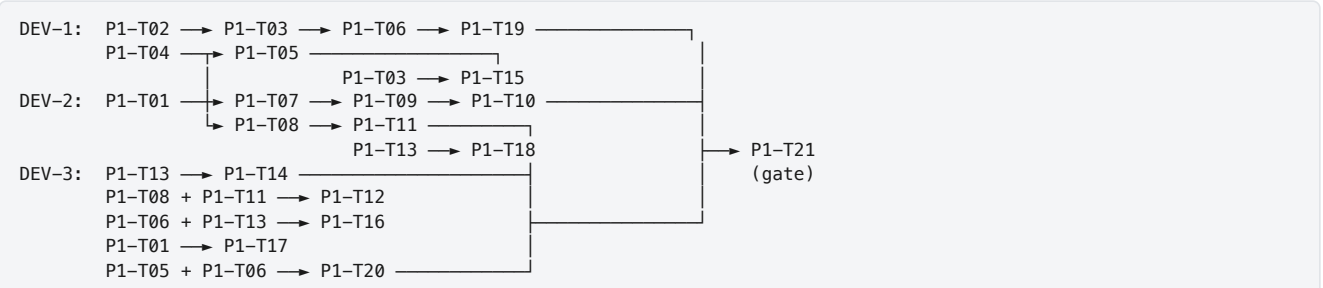
| | | | | | |
|---|---|---|---|---|---|
| **P1-T05** | CI Pipeline Enhancement | Extend `.github/workflows/ci.yml`: (1) add `pylint-odoo` and XML validation alongside flake8, (2) Docker build smoke test — start the stack and assert HTTP 200 on `/web/login`, (3) Odoo test runner — start PostgreSQL service, run `odoo-bin --test-enable -d test_db --stop-after-init -i ncollection_subscription,ncollection_core,ncollection_branding`, (4) dependency + image security scanning (`pip-audit`, `trivy`) as non-blocking warnings initially, (5) document branch protection (require CI + 1 approval on `develop`).<br>**Reason**: flake8 alone cannot catch broken views, missing dependencies, failing model logic, or vulnerable dependencies.<br>**Risks**: CI time grows to 5–8 minutes — run lint and build jobs in parallel.<br>**Acceptance**: a PR that breaks a model or view fails CI; scan reports appear on every PR. | `[DEV-1]` | P1-T01, P1-T04 | 2 |
| **P1-T06** | DB Routing Engine & Multi-DB Verification | Prove end-to-end subdomain→database routing. (1) Verify `db_filter=^%d$` against multiple databases, (2) create test databases `clienta` and `clientb` plus the `admin` database, (3) dev workaround: `/etc/hosts` entries for `clienta.localhost`, `clientb.localhost`, `admin.localhost` and a matching Nginx dev config, (4) verify session isolation — a login on `clienta` must not carry to `clientb`, (5) document the complete routing flow with a diagram and troubleshooting section in `docs/ROUTING.md`.<br>**Reason**: this is the backbone of the platform — nothing ships until routing is bulletproof.<br>**Acceptance**: each subdomain reaches only its own database; the database selector is unreachable; sessions do not leak across tenants. | `[DEV-1]` | P1-T03 | 4 |
| **P1-T07** | Tenant & Subscription Model Enhancements | Enhance the EXISTING `ncollection.tenant` / `ncollection.subscription` / `ncollection.subscription.plan` models with the business logic the workspace needs. (1) Add `allowed_module_names` Text field on the plan (comma-separated technical names like `crm,sale,stock` — deliberately NOT a Many2many to `ir.module.module`, to avoid cross-database coupling), (2) `days_remaining` computed field on subscription, (3) state transition methods `action_activate/suspend/expire/cancel/renew` that validate the current state before transitioning, (4) `_check_` constraints (`end_date > start_date`, `max_users > 0`), (5) `_track_subtype` chatter notifications on status change, (6) unit tests for every transition and constraint.<br>**Reason**: the models exist but have no guarded lifecycle and no module licensing data — both are prerequisites for visibility, enforcement, and provisioning.<br>**Acceptance**: invalid transitions raise ValidationError; the plan stores a module list; tests pass in CI. | `[DEV-2]` | P1-T01 | 5 |
| **P1-T08** | Tenant Role Definitions | Create XML data in `ncollection_core` defining `res.groups` for the 8 standard tenant roles: Owner (full control incl. billing), CEO (all modules, read-only financials), Manager (department-level), Sales (implies `sales_team.group_sale_salesman`), Warehouse (implies `stock.group_stock_user`), HR (implies `hr.group_hr_user`), Accountant (implies `account.group_account_user`), Employee (self-service only). Use `implied_ids` chains carefully — document every inheritance decision in a role matrix (`docs/ROLE_MATRIX.md`).<br>**Reason**: predefined roles are what make onboarding an SMB possible without teaching them Odoo's permission system.<br>**Risks**: `implied_ids` chains can silently escalate permissions — the matrix doc plus P1-T21 testing guard against this.<br>**Acceptance**: all groups importable; each role sees exactly the menus/actions defined in the matrix. | `[DEV-2]` | P1-T01 | 3 |

| | | | | | |
|---|---|---|---|---|---|
| **P1-T09** | Module Visibility Engine (Menus) | Build the UI layer of subscription-based module control. (1) Create `ncollection.workspace.config` model (one record per tenant DB, written during provisioning): `allowed_module_names`, `plan_code`, `subscription_status`, (2) override `ir.ui.menu._visible_menu_ids()` in `ncollection_core` to remove menus of unlicensed modules, (3) respect Odoo's menu caching and clear caches on config change, (4) test with two plans: Starter (`crm,sale,account`) vs Enterprise (all).<br>**Reason**: subscription-based module visibility is THE core product differentiator.<br>**Risks**: `_visible_menu_ids` is an internal method — pin the override to the Odoo 19 signature and add a startup assertion.<br>**Note**: this task is deliberately UI-ONLY; the security layer is P1-T10 — the two together form defense in depth.<br>**Acceptance**: a Starter tenant sees only licensed menus; changing workspace config updates menus after cache clear. | `[DEV-2]` | P1-T07 | 4 |
| **P1-T10** | License Enforcement at ORM & RPC Layer | Menu hiding is a UI convenience, NOT security — a savvy user can hit unlicensed models via direct URLs or XML-RPC/JSON-RPC. Enforce licensing at the data layer: (1) map licensed module set → allowed model namespaces, (2) generate/activate `ir.rule` record rules and/or override `check_access_rights` via an AbstractModel mixin so read/write/create/unlink on models of unlicensed modules is denied for all non-system users, (3) return a branded "not in your plan" error (upsell message) instead of a raw AccessError where the UI can catch it, (4) automated tests: RPC calls against unlicensed models must be denied for a Starter tenant and allowed for Enterprise, (5) measure ORM overhead — enforcement must add < 5ms per request.<br>**Reason**: without ORM-level enforcement, module licensing is trivially bypassable — a paying-for-Starter tenant could use the full ERP via API.<br>**Acceptance**: the P1-T20 E2E suite proves URL and RPC access to unlicensed modules is blocked. | `[DEV-2]` | P1-T09 | 3 |
| **P1-T11** | Apps & Settings Menu Stripping | Lock tenant users out of platform-dangerous areas. (1) Restrict the Apps menu (`base.menu_management`) and Settings (`base.menu_administration`) to the Owner group, (2) return 403 from the Settings controller for non-Owner users, (3) block `/web/database/manager` and `/web/database/selector` at BOTH Nginx and Odoo levels, (4) block debug mode activation (`?debug=1`) for non-Owner users.<br>**Reason**: a tenant user who uninstalls a module or edits system parameters creates a support incident — or an outage.<br>**Acceptance**: a Sales-role user sees no Apps/Settings menus and every direct URL returns Access Denied; the database manager is unreachable from any tenant subdomain. | `[DEV-2]` | P1-T08 | 2 |
| **P1-T12** | Owner Workspace Settings & User Management | Build the simplified admin surface a tenant Owner actually needs (instead of raw Odoo Settings, which P1-T11 strips). (1) "Workspace Settings" menu visible to Owner only: company info (name, logo, address, TRN), user management (invite user by email, assign one of the 8 NCollection roles, deactivate user), (2) max-user enforcement against the subscription limit with a friendly upgrade hint when the limit is reached, (3) do NOT expose raw `res.groups` — role selection maps to the predefined groups from P1-T08.<br>**Reason**: without a safe replacement for Settings, every tenant user change becomes a support ticket.<br>**Acceptance**: Owner can invite/deactivate users and assign roles; a user beyond the plan limit is blocked with a clear message; non-Owner roles cannot see the menu. | `[DEV-3]` | P1-T08, P1-T11 | 3 |
| **P1-T13** | Web Client Branding Completion | Finish the EXISTING `ncollection_branding` module. (1) Navbar logo replacement (OWL patch or QWeb inheritance), (2) loading/splash screen, (3) "Powered by Odoo" footer overrides, (4) About dialog replacement (NCollection version + copyright), (5) backend wallpaper, (6) branded 404/500 error pages, (7) full audit: grep the rendered frontend for "Odoo" — every visible occurrence replaced or hidden, (8) document LGPL v3 attribution obligations in `docs/LEGAL.md` — rebranding the UI is permitted but source attribution rules must be recorded.<br>**Reason**: one visible "Odoo" string breaks the white-label promise.<br>**Risks**: some strings live deep in JS bundles — override at template level where possible, CSS-hide as a last resort, and log unfixable cases.<br>**Acceptance**: zero visible Odoo references in a full click-through of every licensed module. | `[DEV-3]` | None | 5 |

| | | | | | |
|---|---|---|---|---|---|
| **P1-T14** | Login Page Redesign | Redesign `/web/login` via QWeb inheritance on `web.login` — template ONLY, never the auth controller logic (preserves CSRF and session security). (1) Full-page branded background, (2) NCollection logo and centered login card, (3) Remember Me, (4) functional Forgot Password link, (5) copyright footer, (6) mobile responsive, (7) optional per-tenant logo detection by subdomain, (8) zero Odoo references in the page source.<br>**Reason**: the login page is the first impression of the product.<br>**Acceptance**: pixel-reviewed at 320/768/1440 widths; login, remember-me, and password reset all function. | `[DEV-3]` | P1-T13 | 2 |
| **P1-T15** | Public URL Rewriting (Scoped) | TIMEBOXED (1 day), reassigned to DEV-1 because this is pure Nginx work. Hide "odoo" from PUBLIC-FACING URLs only: login, password reset, portal, and website pages. Odoo 19's JavaScript router hardcodes internal `/odoo/...` backend paths — rewriting them causes endless routing bugs on every upgrade, so internal backend URLs stay as-is (accepted trade-off, documented). (1) Nginx rewrites + 301s for public surfaces, (2) verify assets, websocket, and redirects still work, (3) document the decision boundary in `docs/ROUTING.md`.<br>**Reason**: white-labeling matters most where prospects and portal users look; breaking the backend router for cosmetic URLs is a bad trade.<br>**Acceptance**: no "odoo" appears in any public/portal/login URL; the backend still functions flawlessly. | `[DEV-1]` | P1-T03 | 1 |
| **P1-T16** | Dynamic Tenant Branding | Per-tenant visual customization. (1) Extend `res.company`: `nc_primary_color`, `nc_secondary_color`, `nc_sidebar_color`, `nc_login_background` (Image), (2) inject a style block with CSS custom properties (`--nc-primary`, etc.) on page load, (3) refactor `ncollection_branding` SCSS to consume the variables with NCollection defaults as fallback, (4) "Workspace Appearance" page for the Owner (integrates into the P1-T12 settings menu) to pick colors and upload a logo, (5) server-side validation of color values (strict hex format) to prevent CSS/XSS injection.<br>**Reason**: tenant-owned branding is a premium differentiator and drives perceived value.<br>**Acceptance**: changing a color updates navbar/sidebar on reload; invalid color strings are rejected server-side. | `[DEV-3]` | P1-T06, P1-T13 | 4 |
| **P1-T17** | Customer Workspace Dashboard | Build the tenant-facing landing dashboard in OWL (this is the CUSTOMER dashboard — the SaaS admin dashboard already exists and is a different thing). Widgets: sales this month vs last (trend arrow), receivables, payables, cash/bank balance, open activities, pending approvals, quick actions (New Quotation / Invoice / PO), 6-month revenue line chart, top-5 customers bar chart. Data via ORM `read_group` / `search_count` through a dedicated backend service model — optimize to SQL only if measured slow. Role-aware: Accountant sees financial widgets, Sales sees pipeline widgets, CEO sees everything (driven by the P1-T08 groups). Responsive down to tablet.<br>**Reason**: landing on a KPI dashboard instead of an app grid is what makes this feel like a product.<br>**Acceptance**: loads under 2s on demo data; widgets respect roles; charts render with the tenant's brand colors. | `[DEV-3]` | P1-T01 | 5 |
| **P1-T18** | Email Template Branding | Create one branded base email layout (logo header, brand accents, footer with company info) and rebase all transactional templates on it: password reset, user invitation, quotation/SO, invoice, purchase order. Remove every Odoo reference from email HTML. Test rendering in Gmail and Outlook (mobile + desktop).<br>**Reason**: emails are a high-visibility touchpoint and currently leak Odoo branding. Reassigned to DEV-2 (QWeb/XML-heavy work) to relieve the DEV-3 bottleneck identified in the planning review.<br>**Risks**: Odoo email templates use layered QWeb inheritance — override at the highest shared layout (`mail.mail_notification_layout`) to survive upgrades.<br>**Acceptance**: every listed template sends with NCollection branding and renders correctly in both clients. | `[DEV-2]` | P1-T13 | 3 |

| | | | | | |
|---|---|---|---|---|---|
| **P1-T19** | Authentication Hardening (OCA-First) | Harden authentication WITHOUT overriding Odoo's login controller wholesale (fragile across versions — Rule 2 applies). (1) Evaluate and install OCA `auth_brute_force` (login rate limiting/lockout) and `auth_session_timeout` from `OCA/server-auth`; if no Odoo 19 port exists, port the OCA module (preferred) or implement the minimal equivalent behind a feature flag — document the decision, (2) `ncollection.auth.log` model recording login success/failure, logout, password resets (IP, user agent, database), (3) configurable session timeout via `ir.config_parameter`, (4) verified secure cookie flags (`Secure`, `HttpOnly`, `SameSite=Lax`), (5) verify Odoo's password reset tokens are time-limited and single-use.<br>**Reason**: default Odoo auth has no brute-force protection or audit trail — table stakes for commercial SaaS; OCA modules eliminate the lockout-bug risk of custom auth code.<br>**Acceptance**: repeated failed logins trigger lockout; all auth events appear in the log; Nginx (edge) + OCA (app) rate limits both verified. | `[DEV-1]` | P1-T06 | 3 |
| **P1-T20** | E2E Test Framework (Playwright) | Stand up the automated end-to-end testing foundation that replaces unsustainable manual multi-tenant testing. (1) Playwright project with fixtures that spin up two test tenant DBs on different plans, (2) core journeys: login/logout per subdomain, session isolation (login A, visit B → login page), module visibility per plan, RPC license-enforcement probes (with P1-T10), role menu matrix spot checks, branding audit (page source contains no "Odoo"), (3) wire into CI (P1-T05) to run on every PR to `develop`, (4) document how to add new journeys.<br>**Reason**: by Phase 3 there will be 8 roles × multiple plans × multiple tenants — manual verification would take weeks and rot instantly; automation makes the isolation guarantee continuously enforced.<br>**Acceptance**: the suite runs green in CI in under 10 minutes and fails when an isolation or visibility regression is introduced deliberately. | `[DEV-3]` | P1-T05, P1-T06 | 3 |
| **P1-T21** | Phase 1 Integration Testing & Security Audit | The Phase 1 gate. With 2+ test tenants on different plans: (1) run the full P1-T20 E2E suite plus exploratory testing, (2) role click-through matrix for all 8 roles, (3) scripted cross-tenant RPC attack attempts MUST fail, (4) license bypass attempts (URL + RPC) MUST fail, (5) branding audit: zero Odoo references, (6) auth: lockout, session timeout and reset flows, (7) dashboard data correctness per role, (8) email rendering, (9) publish the regression checklist that every future phase re-runs, (10) publish a test report in `docs/`.<br>**Reason**: features that pass alone fail together — isolation and role gaps only appear under combined testing.<br>**Acceptance**: signed-off test report; regression checklist committed; all criticals fixed or ticketed. | `[DEV-1]` | P1-T06, P1-T10, P1-T11, P1-T12, P1-T14, P1-T15, P1-T16, P1-T17, P1-T18, P1-T19, P1-T20 | 3 |

## Phase 1 Dependency Graph




**Day-1 parallel starts** (no cross-blocking): DEV-1 → P1-T02 + P1-T04, DEV-2 → P1-T01 (then P1-T07/T08), DEV-3 → P1-T13.

## Phase 1 Developer Workload

| Developer | Tasks | Total Days | Notes |
|---|---|---|---|
| DEV-1 | T02, T03, T04, T05, T06, T15, T19, T21 | 20 | Infra chain is the critical path (16 days sequential) |
| DEV-2 | T01, T07, T08, T09, T10, T11, T18 | 21 | Starts Day 1 via T01 (no infra dependency) |

| | | | |
|---|---|---|---|
| DEV-3 | T12, T13, T14, T16, T17, T20 | 22 | Rebalanced from 23 (v4.0): email → DEV-2, URL rewriting → DEV-1; gained E2E + Owner settings |

**Critical path**: `P1-T02 → P1-T03 → P1-T06 → P1-T19 → P1-T21` (DEV-1, 15 days) and `P1-T01 → P1-T07 → P1-T09 → P1-T10 → P1-T21` (DEV-2, 16 days) — Phase 1 cannot complete in fewer than **~16 working days** of pure development plus buffer → 8–10 calendar weeks including reviews, iteration, and stabilization.

---

## Phase 2: SaaS Automation

**Priority**: P1 — HIGH (starts after the Phase 1 gate) **Objective**: Automate tenant provisioning, billing, payment collection, subscription lifecycle, backups (PITR), domains, and stand up staging + production operations.

> DEV-1 is intentionally the bottleneck of this phase (it is infrastructure-dominated). DEV-2 and DEV-3 begin Phase 3 tasks (UAE localization, translations, PDF invoices) in parallel once their Phase 2 queue drains.

**Acceptance Criteria (Definition of Done)**:

- [ ] A stranger can self-provision a trial workspace end-to-end with zero staff involvement
- [ ] Subscription activation → provisioned tenant in under 10 minutes, fully automated, idempotent on retry
- [ ] Plan changes propagate to the tenant workspace within one minute
- [ ] PITR can restore the cluster to any minute in the last 7 days; tenant-level restore drill passes
- [ ] Subscriptions can be paid online (Stripe test + live mode)
- [ ] Staging auto-deploys on merge to `develop`; production servers are hardened; uptime alerting is live

### Phase 2 Tasks

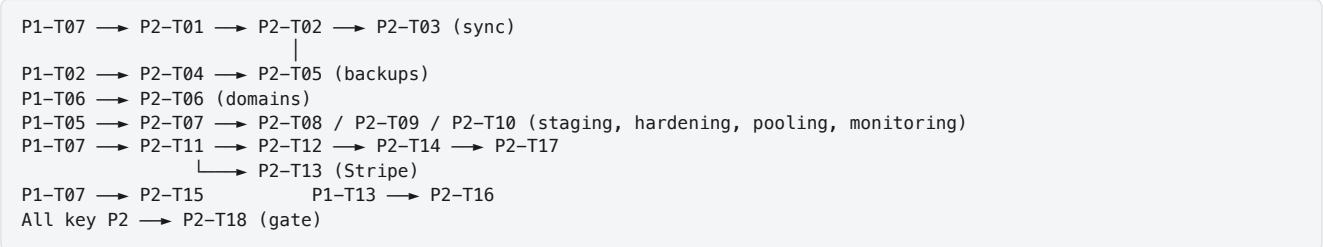
| ID | Task Name | Description | Assigned | Dependencies | Est. Days |
|---|---|---|---|---|---|
| **P2-T01** | Dedicated Provisioning Runner & Engine Core | Turn the EXISTING `ncollection.provisioning.job` model into a working engine executed in an ISOLATED runner. (1) Add a dedicated Docker container running OCA `queue_job` workers — provisioning (CREATE DATABASE + `odoo-bin -d {db} -i base --stop-after-init` + module installs) is CPU/RAM intensive and must NEVER run on the HTTP workers serving live tenants, (2) engine steps: create tenant DB, install the plan's modules from `allowed_module_names`, create tenant admin user with forced password reset, write the `ncollection.workspace.config` record, apply branding defaults, (3) step-by-step status + log updates on the job, (4) rollback (drop DB) on failure. Evaluate `queue_job` 19.0 availability FIRST; document the fallback (subprocess with resource limits) if unavailable.<br>**Reason**: manual DB creation cannot scale past a handful of tenants, and in-process provisioning would cause latency spikes for paying customers.<br>**Acceptance**: a queued job produces a login-ready tenant DB (or a clean rollback with readable logs) while a load test against existing tenants shows no latency degradation. | `[DEV-1]` | P1-T07 | 5 |
| **P2-T02** | Auto-Provisioning Pipeline | End-to-end automation: subscription `draft→active` (or `trial`) triggers provisioning without human action. (1) Sanitized, collision-safe DB name generation from company name, (2) job creation + async execution on the dedicated runner, (3) on success: tenant status active, `portal_url` set, welcome email fired, (4) on failure: admin alert with the job log, (5) idempotent — safe to retry a half-failed provision, (6) "Provision Now" manual button retained on the tenant form.<br>**Acceptance**: activating a subscription yields a working tenant in under 10 minutes with zero manual steps; retrying a deliberately failed job heals cleanly. | `[DEV-1]` | P2-T01 | 4 |

| | | | | | |
|---|---|---|---|---|---|
| **P2-T03** | Workspace Config Sync & Plan Change Propagation | Keep tenant workspaces in sync with the platform. When a subscription changes (plan upgrade/downgrade, suspension, renewal), push the new `allowed_module_names` and `subscription_status` into the tenant DB's `ncollection.workspace.config`. Mechanism: XML-RPC call to localhost with the tenant DB name using a dedicated service account (credentials secured per ARCHITECTURE_SECURITY.md) — never direct cross-DB SQL. Trigger on write + a nightly reconciliation cron that repairs drift. Suspended tenants get a branded "Subscription Expired" interstitial.<br>**Reason**: without propagation, plan changes silently never reach the tenant — billing and access desync.<br>**Acceptance**: a plan upgrade adds menus (and ORM access) in the tenant workspace within one minute; the reconciliation cron heals a manually broken config. | `[DEV-2]` | P2-T02 | 3 |
| **P2-T04** | PITR & WAL Archiving (pgBackRest) | Daily dumps alone mean a 24-hour RPO — unacceptable for commercial SaaS. Implement Point-in-Time Recovery: (1) deploy `pgBackRest` (or WAL-G — evaluate, document choice) against the PostgreSQL cluster with continuous WAL archiving to S3/Backblaze B2, (2) full base backup weekly + differentials daily, (3) encryption of archives at rest, (4) retention policy, (5) documented + scripted restore procedure including the tenant-level nuance: PITR restores the CLUSTER to a point in time — per-tenant restore means restoring to a scratch instance and extracting the single DB (see ARCHITECTURE_DATA_PLATFORM.md §5), (6) restore rehearsal on staging.<br>**Reason**: with PITR the RPO drops from 24 hours to ~1 minute — a tenant corrupting data at 4 PM loses minutes, not a business day.<br>**Acceptance**: demonstrated restore of the staging cluster to an arbitrary timestamp; WAL archive lag alerting configured. | `[DEV-1]` | P1-T02 | 3 |
| **P2-T05** | Tenant Backup Manager & Restore Drills | Tenant-granular backup layer on top of PITR: (1) nightly `pg_dump --format=custom` per active tenant DB PLUS tar of the tenant filestore (attachments live on disk, not in the DB), (2) compress + encrypt, upload to S3/Backblaze B2, (3) retention 7 daily / 4 weekly / 12 monthly, (4) `ncollection.backup` records with results and alerting on failure, (5) "Restore Backup" wizard to a staging DB, (6) monthly restore drill scheduled and documented.<br>**Reason**: pg_dump gives cheap per-tenant restore granularity and long-term archival; PITR (P2-T04) covers disaster recovery — together they satisfy both RPO and per-tenant needs.<br>**Acceptance**: backup of a demo tenant restores to a working workspace including attachments. | `[DEV-1]` | P2-T04 | 4 |
| **P2-T06** | Domain & SSL Automation | Automate domains and certificates. On provisioning: (1) render the Nginx server block from a Jinja2 template, (2) graceful nginx reload, (3) tenant subdomains ride the wildcard cert (per-tenant Let's Encrypt only for future custom domains), (4) `ncollection.domain` model tracking domain + SSL expiry, (5) weekly renewal cron with 14-day-lead alerting.<br>**Acceptance**: a newly provisioned tenant is reachable over HTTPS with zero manual server work. | `[DEV-1]` | P1-T06 | 3 |
| **P2-T07** | Staging Environment & Continuous Deployment | Stand up the staging server and continuous deployment — promised by the tooling guide but never tasked in v4.0. (1) Provision the Hetzner VPS (Ubuntu 24.04, Docker), (2) production compose files running against real DNS (`*.staging.ncollectionerp.com`), (3) GitHub Actions: on merge to `develop` — build image, push to registry, SSH deploy, health-check smoke test, Discord notification, (4) documented rollback (previous image tag).<br>**Reason**: every later phase needs multi-tenant behavior tested on real DNS + TLS; deploys must be boring by go-live.<br>**Acceptance**: merging to `develop` updates staging automatically within 10 minutes. | `[DEV-1]` | P1-T05 | 3 |
| **P2-T08** | Production Server Hardening | Harden the server(s): UFW (only 22/80/443), SSH key-only auth + fail2ban, PostgreSQL never exposed publicly, Docker socket protection, unattended security updates, secrets outside git with restricted permissions, least-privilege deploy user. Produce `docs/RUNBOOK_SECURITY.md`.<br>**Reason**: the v4.0 security table listed firewall/SSH hardening as "Phase 2" but no task implemented it.<br>**Acceptance**: external port scan shows only 22/80/443; SSH password auth rejected; checklist committed. | `[DEV-1]` | P2-T07 | 2 |

| | | | | | |
|---|---|---|---|---|---|
| **P2-T09** | Connection Pooling Topology (PgBouncer) | Deploy PgBouncer with the topology from ARCHITECTURE_DATA_PLATFORM.md §4: (1) transaction-pooling pool for Odoo HTTP workers (port 8069 traffic) sized against `max_connections`, (2) Odoo's longpolling/bus (port 8072) and the cron + queue_job workers connect DIRECT to PostgreSQL — LISTEN/NOTIFY and long-lived sessions break under transaction pooling, (3) per-database pool limits so one hot tenant cannot starve others, (4) monitoring hooks (pool saturation).<br>**Reason**: connection count grows linearly with tenants; without pooling the cluster falls over around ~20 active tenants — and naive pooling breaks Odoo's realtime bus.<br>**Acceptance**: staging runs all traffic through the correct paths; chatter/bus notifications still work; pool metrics visible. | `[DEV-1]` | P2-T07 | 2 |
| **P2-T10** | Platform Uptime Monitoring & Alerting | Lightweight production monitoring NOW (the full Prometheus stack lands in Phase 8). (1) Uptime Kuma (or Healthchecks.io) probing each tenant subdomain + admin, (2) disk/memory/CPU threshold alerts via a simple agent or cron script, (3) Odoo ERROR-level log watcher, (4) WAL-archive and backup-failure alerts, (5) all alerts to Discord.<br>**Reason**: the first production tenants arrive at the end of this phase — flying blind until Phase 8 is not acceptable.<br>**Acceptance**: killing the Odoo container triggers a Discord alert within 2 minutes. | `[DEV-1]` | P2-T07 | 2 |
| **P2-T11** | Billing Engine | Automatic invoicing for subscriptions in the admin DB. (1) `account.move` generation on purchase and renewal (plan, period, price), (2) UAE VAT 5% applied, (3) proration on mid-cycle upgrades, (4) invoice linked to tenant + subscription, (5) payment status tracked on the subscription. CHECK OCA first (contract/subscription-oriented modules) and document the decision.<br>**Acceptance**: activating or renewing a subscription always produces exactly one correct invoice. | `[DEV-2]` | P1-T07 | 5 |
| **P2-T12** | Subscription Lifecycle & Trial Support | Formal lifecycle: `draft→trial→active→expired→suspended→terminated` (+ `active→cancelled`). Trial support: `trial_days` on plan, trial state with full plan access, auto-conversion or expiry at trial end. Transition side effects: activation → provision + invoice; expiry → grace period; suspension → access blocked with the branded interstitial (via P2-T03); reactivation flow. Guarded transitions with constraints + tests.<br>**Acceptance**: every transition path tested, including trial conversion and reactivation during grace. | `[DEV-2]` | P2-T11 | 4 |
| **P2-T13** | Subscription Payment Collection (Stripe) | Collect subscription money online — v4.0 deferred ALL payment capability to Phase 6, meaning go-live with manual bank-transfer chasing. Configure Odoo's built-in `payment_stripe` provider in the admin DB (Community ships it — do NOT build a gateway from scratch): (1) payment links / hosted checkout attached to subscription invoices and renewal emails, (2) webhook confirmation marks the invoice paid and extends the subscription, (3) failed-payment handling feeds the dunning scheduler. Regional gateways (PayTabs, Tap) are deliberately Phase 6 scope (tenant-facing).<br>**Acceptance**: a test-mode card payment marks the invoice paid and renews the subscription automatically. | `[DEV-2]` | P2-T11 | 4 |
| **P2-T14** | Expiration & Dunning Scheduler | Daily lifecycle cron: (1) advance-warning emails 30/14/7/1 days before expiry, (2) transition to `expired` after `end_date` (+48h safety buffer), (3) suspension after the 15-day grace, (4) failed-payment dunning sequence (retry schedule + emails), (5) admin override to reactivate, (6) every action logged to chatter.<br>**Acceptance**: simulated-clock tests prove each threshold fires exactly once per subscription. | `[DEV-2]` | P2-T12 | 2 |
| **P2-T15** | SaaS Admin Dashboard Enhancement | Extend the EXISTING SaaS admin dashboard: KPI cards (tenants, MRR, churn, trial conversions), tenant list with status badges and quick actions, provisioning job monitor, revenue trend + per-plan breakdown, DB size / storage health. Restrict to a new `NCollection / Platform Admin` group.<br>**Acceptance**: platform staff can spot a failed provision or an expiring tenant in under 10 seconds. | `[DEV-2]` | P1-T07 | 4 |

| | | | | | |
|---|---|---|---|---|---|
| **P2-T16** | Self-Service Onboarding & Public Checkout | The platform's best sales tool: visit the site, pick a plan, get an isolated ERP in minutes. (1) Pricing page with plan comparison + monthly/yearly toggle, (2) company registration form (validated, reCAPTCHA, unique company/subdomain check with live availability feedback), (3) creates tenant + trial/draft subscription and triggers provisioning, (4) "your workspace is being prepared" progress page that polls provisioning status and reveals the login URL when ready, (5) NCollection branding, zero Odoo references, bilingual-ready copy. **Acceptance**: a stranger can self-provision a trial workspace end-to-end without staff, in under 10 minutes. | `[DEV-3]` | P1-T13 | 5 |
| **P2-T17** | Email Automation System | Complete the transactional email set on the P1-T18 base layout: welcome (login URL + getting started), trial ending, renewal reminders, expiration, suspension warning, payment received, payment failed, plan change confirmations. De-duplicate so a tenant never receives two lifecycle emails on the same day. **Acceptance**: every lifecycle transition sends exactly the right email, verified on staging. | `[DEV-3]` | P1-T18, P2-T14 | 3 |
| **P2-T18** | Phase 2 Integration & E2E Suite Expansion | The Phase 2 gate. End-to-end on staging: public checkout → provision → login → pay (Stripe test mode) → plan upgrade (config sync visible) → backup → tenant restore → suspend → reactivate. Extend the Playwright suite (P1-T20) with checkout and lifecycle journeys. Re-run the full Phase 1 regression checklist. **Acceptance**: signed-off test report; the full tenant lifecycle needs zero manual server intervention; E2E suite green in CI. | `[DEV-1]` | P2-T02, P2-T03, P2-T05, P2-T06, P2-T13, P2-T16, P2-T17 | 3 |

### Phase 2 Dependency Graph




### Phase 2 Developer Workload

| Developer | Tasks | Total Days | Notes |
|---|---|---|---|
| DEV-1 | T01, T02, T04, T05, T06, T07, T08, T09, T10, T18 | 31 | Deliberate bottleneck — infra phase |
| DEV-2 | T03, T11, T12, T13, T14, T15 | 22 | Starts Phase 3 (UAE) when queue drains |
| DEV-3 | T16, T17 | 8 | Starts Phase 3 (translations, PDF) early |

---

## Phase 3: ERP Enhancement & UAE Localization

**Priority**: P1 — HIGH (overlaps Phase 2 for DEV-2 and DEV-3) **Objective**: Full UAE/GCC localization, core ERP polish, performance baselines, pre-launch security assessment — ending in the **first production deployment**.

**Acceptance Criteria (Definition of Done)**:

- [ ] A provisioned tenant computes 5% UAE VAT with the UAE Chart of Accounts out of the box
- [ ] Fully bilingual Arabic/English interface with correct RTL rendering
- [ ] UAE-compliant bilingual PDF invoices with TRN and QR code
- [ ] Documented performance baseline under simulated multi-tenant load
- [ ] External-grade security assessment passed; go-live checklist executed with evidence
- [ ] Real paying tenant(s) live in production

## Phase 3 Tasks

| ID | Task Name | Description | Assigned | Dependencies | Est. Days |
|---|---|---|---|---|---|
| **P3-T01** | OCA Financial Stack Verification | Verify the EXISTING OCA modules ( `account_financial_report` , `mis_builder` ) install cleanly during automated provisioning and function inside tenant DBs. Add them to the provisioning module set for plans that include accounting. Test every report against UAE demo data. Pin versions in `repos.yml` (P1-T04).<br>**Acceptance**: a freshly provisioned Enterprise tenant can run a Trial Balance immediately. | `[DEV-2]` | P2-T01 | 2 |
| **P3-T02** | PostgreSQL Performance Tuning | Tune `postgresql.conf` for multi-tenant load per ARCHITECTURE_DATA_PLATFORM.md §9: `shared_buffers` (25% RAM), `effective_cache_size` (75%), `work_mem` , `maintenance_work_mem` , SSD-appropriate settings. Enable `pg_stat_statements` ; log queries over 500ms. Baseline before/after with the same workload.<br>**Acceptance**: documented measurable improvement and a committed config template. | `[DEV-1]` | P2-T07 | 2 |
| **P3-T03** | Odoo Worker Tuning & Load Testing | Tune Odoo workers from measured load: worker count vs memory reality, cron/queue worker isolation, `limit_time` tuning. Load-test with k6/Locust simulating 50 concurrent users across 3 tenant DBs on staging (through the PgBouncer topology). Document scaling thresholds (when to add RAM / workers / a second node).<br>**Acceptance**: performance baseline doc with graphs committed to `docs/` . | `[DEV-1]` | P3-T02, P2-T09 | 2 |
| **P3-T04** | UAE VAT Configuration | Create the `ncollection_uae` addon with UAE VAT: 5% standard, 0% zero-rated, exempt; tax groups; fiscal positions for domestic/GCC/international; defaults wired to products and accounts. All XML data — installable unattended during provisioning.<br>**Acceptance**: a sale in a fresh tenant computes 5% VAT with correct tax accounts. | `[DEV-2]` | P1-T01 | 3 |
| **P3-T05** | UAE Chart of Accounts | UAE CoA in `ncollection_uae` as an account chart template: Assets 1xxx, Liabilities 2xxx, Equity 3xxx, Revenue 4xxx, COGS 5xxx, Expenses 6xxx–7xxx, VAT accounts linked to P3-T04 taxes. Test the full sale→invoice→payment→reconciliation cycle.<br>**Acceptance**: journal entries post to correct accounts through the whole cycle. | `[DEV-2]` | P3-T04 | 4 |
| **P3-T06** | AED & Multi-Currency Setup | AED default currency, multi-currency activation, automated exchange rates (UAE Central Bank or ECB provider — check `OCA/currency` first), GCC currencies preloaded (USD, EUR, SAR, KWD, BHD, QAR, OMR), UAE rounding rules.<br>**Acceptance**: a USD invoice posts with correct AED conversion at the day's rate. | `[DEV-2]` | P3-T04 | 2 |
| **P3-T07** | Approval Workflow Enhancements | Configurable approval workflows without touching core: sales orders above a threshold require manager approval; purchases require two-level approval (department + finance); CRM territory-based lead assignment. Implemented with `mail.activity` + state fields in a custom addon.<br>**Acceptance**: a threshold-crossing SO cannot confirm until the approval activity completes. | `[DEV-2]` | P1-T08 | 5 |
| **P3-T08** | Arabic/English Translation & RTL Audit | Full bilingual pass: export `.po` files for every `ncollection_*` module, professional Arabic translation of labels/menus/status/errors, RTL layout audit of backend + dashboard + login, Arabic PDF rendering (embedded font support).<br>**Acceptance**: switching to Arabic yields a fully translated RTL interface with zero broken layouts on core flows. | `[DEV-3]` | P1-T13 | 5 |
| **P3-T09** | UAE-Compliant PDF Invoice Templates | UAE-compliant QWeb invoice PDFs: bilingual Arabic/English layout, TRN display, itemized VAT summary, QR code (e-invoicing readiness), sequential numbering, bank details, tenant branding integration (P1-T16 colors/logo), A4 + thermal formats.<br>**Acceptance**: a sample invoice passes a UAE tax-invoice requirements checklist (documented in the PR). | `[DEV-3]` | P3-T04 | 4 |

| | | | | | |
|---|---|---|---|---|---|
| **P3-T10** | MIS Builder Report Enhancement | Extend `ncollection_mis_templates` : corrected Balance Sheet grouping for the UAE CoA, P&L, cash flow if feasible, period comparison, budget vs actual. **Acceptance**: reports reconcile to the penny with the General Ledger on demo data. | `[DEV-3]` | P3-T05 | 3 |
| **P3-T11** | Tenant Data Import Toolkit | Onboarding import toolkit: documented CSV/XLSX templates + import wizards for customers, suppliers, products, opening stock, and opening balances. Validation with row-level error reporting a non-technical admin can understand. **Reason**: the first real tenants arrive with existing data — without this, onboarding is manual data entry. **Acceptance**: the full template set imports into a fresh tenant without developer help. | `[DEV-2]` | P3-T05 | 3 |
| **P3-T12** | Pre-Launch Security Assessment | Formal security assessment BEFORE real tenant data arrives (v4.0 deferred this to Phase 10 — far too late). (1) Run the full checklist from ARCHITECTURE_SECURITY.md: isolation suite, license enforcement, auth hardening, headers, TLS config (SSL Labs A grade), secrets audit, dependency scan review, (2) OWASP-style probing of checkout, login, portal-facing endpoints (injection, IDOR, CSRF, SSRF), (3) engage an external tester if budget allows — otherwise a structured internal red-team day with documented methodology, (4) remediate all criticals/highs. **Acceptance**: assessment report committed; zero unresolved critical or high findings. | `[DEV-1]` | P2-T18 | 3 |
| **P3-T13** | Go-Live Readiness & First Production Deployment | The go-live gate. Execute a written checklist with evidence linked in the issue: security assessment passed (P3-T12), PITR + tenant backups verified ON PRODUCTION (P2-T04/T05), monitoring + alerting live (P2-T10), UAE compliance sanity check (P3-T04/05/09), full regression + E2E suite green, rollback procedure rehearsed, incident response runbook ( `docs/RUNBOOK_INCIDENTS.md` ) and on-call rotation agreed. Then deploy production and onboard the first real tenant(s). **Acceptance**: production serves a real paying tenant; every checklist item has linked evidence. | `[DEV-1]` | P3-T12, P3-T05, P3-T08, P3-T09 | 2 |

### Phase 3 Developer Workload

| Developer | Tasks | Total Days |
|---|---|---|
| DEV-1 | T02, T03, T12, T13 | 9 |
| DEV-2 | T01, T04, T05, T06, T07, T11 | 19 |
| DEV-3 | T08, T09, T10 | 12 |

---

## Phase 4: Executive Dashboards

**Priority**: P2 — MEDIUM **Objective**: Real-time analytics dashboards for tenant executives and department managers, built on a single reusable aggregation engine.

| ID | Task Name | Description | Assigned | Dependencies | Est. Days |
|---|---|---|---|---|---|
| **P4-T01** | Data Aggregation & Caching Engine | Build the tenant-side aggregation service that powers ALL dashboards (and later the AI context engine): optimized `read_group` /SQL aggregations across sale, account, stock, hr with a caching layer ( `ormcache` or Redis) and cache invalidation on source writes. Establish the query performance budget: every dashboard endpoint under 500ms on 100k-record demo data. **Reason**: every dashboard in Phases 4–5 sits on this engine — build it once, correctly. **Acceptance**: documented aggregation API consumed by P4-T03/T04 with measured query times. | `[DEV-1]` | P1-T07 | 4 |

| | | | | | |
|---|---|---|---|---|---|
| **P4-T02** | KPI Logic Models | `ncollection.kpi` model with computed KPI definitions: Revenue Growth %, Average Deal Size, Days Sales Outstanding, Gross Margin %, Employee Turnover, Inventory Turnover. Each KPI: computation method, period comparison, target/threshold configuration. Unit tests against known fixture data. **Acceptance**: KPI values match hand-calculated fixtures exactly. | `[DEV-2]` | P4-T01 | 3 |
| **P4-T03** | CEO Dashboard UI | CEO dashboard in OWL on the P4-T01 engine: KPI cards with trends, revenue chart, sales pipeline funnel, top customers, date-range selector, drill-down navigation to source records, export to PDF. Loads under 3s. **Acceptance**: renders correctly on desktop/tablet with tenant brand colors and respects role access. | `[DEV-3]` | P4-T02 | 5 |
| **P4-T04** | Department Dashboards | Role-specific dashboards reusing P4-T03 widget components (no copy-paste widgets): Sales (pipeline, targets, leaderboard), Finance (receivables aging, cash position, P&L sparkline), HR (headcount, leave calendar, attendance), Warehouse (stock valuation, low-stock alerts, movement velocity). **Acceptance**: each of the 4 dashboards visible only to its role group. | `[DEV-3]` | P4-T02 | 5 |

---

## Phase 5: AI Platform

**Priority**: P3 — executed AFTER Phase 6 (see §8 execution order) **Objective**: AI-powered assistance for ERP users — behind a single secured gateway, with absolute tenant isolation.

| ID | Task Name | Description | Assigned | Dependencies | Est. Days |
|---|---|---|---|---|---|
| **P5-T01** | LLM Provider Evaluation & Design Spike | Timeboxed spike: evaluate LLM providers (Claude, OpenAI, regional hosting options for data residency), prompt architecture, cost model per tenant, PII-handling policy, streaming vs batch UX. Deliverable: `docs/AI_PLATFORM_DESIGN.md` with the chosen provider, prompt templates, token budget per plan tier, and evaluation results. No production code. **Acceptance**: design doc approved before any AI implementation task starts. | `[DEV-1]` | None | 3 |
| **P5-T02** | LLM Gateway Service | `ncollection_ai` gateway: the single choke point for ALL LLM calls — provider abstraction (swap Claude/OpenAI via config), per-tenant rate limiting and token budgets, request/response logging with PII scrubbing, API keys stored encrypted, circuit breaker on provider outage. **Acceptance**: no module calls an LLM API except through this gateway; budget exhaustion returns a friendly error. | `[DEV-1]` | P5-T01 | 4 |
| **P5-T03** | Context Injection Engine | Builds tenant-scoped context for prompts from ERP data (via P4-T01 aggregations), enforces absolute tenant isolation (the context builder physically cannot read another DB), sanitizes PII per the P5-T01 policy, manages context-window truncation. **Acceptance**: injection tests prove no cross-tenant data can enter a prompt; context quality reviewed on 20 sample questions. | `[DEV-1]` | P5-T02, P4-T01 | 5 |
| **P5-T04** | Anomaly Detection Jobs | Scheduled detection: stock below safety levels, sales trend drops, unusual expense spikes, attendance anomalies. Statistical baselines first (z-scores/moving averages) — LLM explanation layered on top only where useful. `ncollection.alert` records with severity + suggested action, surfaced on dashboards and via email digest. **Acceptance**: seeded anomalies in demo data are detected with zero false negatives on the test set. | `[DEV-2]` | P4-T01 | 4 |
| **P5-T05** | NL→Domain Mapper | Natural-language to Odoo domain translation: the LLM converts user queries (e.g. "unpaid invoices over 5000 AED last quarter") into domain filters for whitelisted models ( `sale.order` , `account.move` , `stock.picking` , `crm.lead` ). CRITICAL: generated domains are parsed and validated server-side against a strict schema — LLM output is NEVER passed to eval or executed raw. **Acceptance**: a 50-question test set passes with domains verified safe; injection attempts in queries produce refusals. | `[DEV-2]` | P5-T02 | 5 |

| | | | | | |
|---|---|---|---|---|---|
| **P5-T06** | AI Chat Widget | Persistent OWL chat widget: floating assistant on every screen, message history per user, markdown rendering, streaming responses, suggested prompts per module context, minimize/expand. Talks only to the P5-T02 gateway. **Acceptance**: usable chat experience demoed across 5 modules with brand styling. | `[DEV-3]` | P5-T03 | 5 |
| **P5-T07** | Smart Search UI | Extend the Odoo command palette/search with a natural-language mode backed by P5-T05: grouped results by model, recent query cache, opt-out toggle. **Acceptance**: NL search returns correct records for the P5-T05 test set from the UI. | `[DEV-3]` | P5-T05 | 4 |

## Phase 6: Customer Portal

**Priority**: P2 — executed BEFORE Phase 5 (portal + regional payments drive revenue and retention) **Objective**: Self-service portal for the tenants' end-customers: invoices, payments, orders, tickets, knowledge base.

| ID | Task Name | Description | Assigned | Dependencies | Est. Days |
|---|---|---|---|---|---|
| **P6-T01** | Regional Payment Gateways (Tenant Invoices) | Payment collection for TENANT invoices (the tenants' end-customers paying them — distinct from the platform's own Stripe billing in P2-T13): PayTabs and Tap Payments provider modules following Odoo's payment-provider framework (reuse patterns from P2-T13), webhook reconciliation, multi-currency, PCI compliance via tokenization (no card data stored). Check `OCA/payment` and existing community providers first. **Acceptance**: an end-customer pays a tenant invoice in AED via the PayTabs sandbox and it auto-reconciles. | `[DEV-1]` | P2-T13 | 5 |
| **P6-T02** | Portal Access Rights | Strict portal isolation: `ir.rule` record rules ensuring portal users see ONLY their own invoices, orders, deliveries, and tickets; penetration-style tests attempting IDOR access to other partners' records via URL manipulation and RPC. **Acceptance**: the isolation test suite (added to E2E) passes; zero cross-partner leakage. | `[DEV-2]` | P1-T08 | 3 |
| **P6-T03** | Support Ticketing | Evaluate `OCA/helpdesk` first; if unsuitable build `ncollection.support.ticket` with portal submission, team assignment rules, SLA timers, stage workflow, email notifications, CSAT rating on close. **Acceptance**: a portal user submits a ticket, an agent resolves it in the backend, the customer rates it — full loop on staging. | `[DEV-2]` | P6-T02 | 5 |
| **P6-T04** | Portal UI Redesign | Override `/my` templates with a modern card-based responsive design carrying the TENANT's branding (P1-T16 colors/logo — the tenant's customers see the tenant's brand, not NCollection's). Bilingual Arabic/English. **Acceptance**: the portal passes the same branding audit standard as the backend (zero Odoo references) and renders RTL correctly. | `[DEV-3]` | P6-T02 | 5 |
| **P6-T05** | Knowledge Base | `ncollection.knowledge.article` with categories, tags, PostgreSQL `tsvector` full-text search (Arabic + English analyzers), WYSIWYG admin editor, view analytics. Portal-facing browse/search UI. **Acceptance**: seeded articles are searchable in both languages from the portal. | `[DEV-3]` | P6-T04 | 4 |

## Phase 7: Mobile Application

**Priority**: P3 **Objective**: Mobile accessibility for field workers and executives — sales entry, approvals, barcode operations, offline capability.

| ID | Task Name | Description | Assigned | Dependencies | Est. Days |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| **P7-T01** | Mobile API Optimization | Mobile-optimized API layer: lightweight JSON endpoints wrapping ORM calls with pagination, sparse field selection, gzip, JWT-based auth (short-lived access + refresh tokens), device registry, API versioning ( `/mobile/v1/` ), rate limiting per device.<br>**Reason**: raw XML-RPC is too chatty for mobile networks.<br>**Acceptance**: documented endpoint set for auth, dashboard, sales, inventory, approvals with median response under 300ms. | `[DEV–1]` | P1-T19 | 5 |
| **P7-T02** | Push Notification Server | FCM integration: device token registration model, notification dispatch service with per-user preferences, triggers for approval requests, lead assignment, stock alerts, payment received.<br>**Acceptance**: an approval request reaches a test device in under 5 seconds. | `[DEV–1]` | P7-T01 | 3 |
| **P7-T03** | Offline Sync Logic | Client sync queue with conflict-resolution policies: last-write-wins for simple fields, server-wins for financial records, append for chatter. Server-side sync journal model with idempotent replay.<br>**Acceptance**: airplane-mode edits sync cleanly on reconnect; a conflicting financial edit is rejected with a clear message. | `[DEV–2]` | P7-T01 | 5 |
| **P7-T04** | Barcode Endpoints | Barcode operations: scan-to-lookup, internal transfer, receipt, picking confirmation — optimized to under 200ms via Redis product cache. Designed for continuous-scanning workflows (no page reload between scans).<br>**Acceptance**: a 50-scan session completes without errors on staging with measured latencies. | `[DEV–2]` | P7-T01 | 4 |
| **P7-T05** | Mobile Framework Decision & App Scaffold | Framework decision (React Native vs Flutter — document the choice and rationale) + app scaffold: navigation shell, login with biometric unlock, secure token storage, API client layer, state management, offline storage foundation, push handler, environment config.<br>**Acceptance**: an authenticated shell app runs on Android and iOS simulators against staging. | `[DEV–3]` | P7-T01 | 5 |
| **P7-T06** | Mobile Core Screens | Core screens: dashboard (KPI cards from the P4 engine), sales order entry, customer lookup, approvals inbox with approve/reject, notifications center, profile/settings. NCollection + tenant branding.<br>**Acceptance**: a sales manager can review KPIs and approve an order end-to-end from the app. | `[DEV–3]` | P7-T05 | 5 |
| **P7-T07** | Mobile Field Operations Screens | Field ops: camera-based barcode scanner for inventory counts/transfers/receipts on the P7-T04 endpoints, offline queue UI showing pending syncs, warehouse task list.<br>**Acceptance**: the receive-scan-confirm warehouse flow works offline and syncs on reconnect. | `[DEV–3]` | P7-T06, P7-T04 | 4 |

## Phase 8: Platform Services

**Priority**: P3 **Objective**: Enterprise integrations (REST API, webhooks, SDKs) and full observability.

| ID | Task Name | Description | Assigned | Dependencies | Est. Days |
|---|---|---|---|---|---|
| **P8-T01** | REST API Foundation | Public REST API foundation: evaluate OCA `base_rest` / the FastAPI addon first — document the choice. OAuth2 client-credentials + authorization-code flows, per-tenant API keys, scoped tokens, rate limiting, request logging, versioned `/api/v1/` routing, standard error envelope.<br>**Acceptance**: the OAuth2 flow issues a token that lists contacts on a demo tenant; unauthorized scopes are rejected. | `[DEV–1]` | P1-T19 | 4 |
| **P8-T02** | REST Business Endpoints | Business endpoints on the P8-T01 foundation: Contacts, Products, Sales Orders, Invoices, Stock levels, CRM leads — full CRUD where sensible, filtered list endpoints with pagination, OpenAPI 3.1 spec auto-generated.<br>**Acceptance**: the OpenAPI spec validates; a Bruno/Postman collection of all endpoints passes against staging. | `[DEV–1]` | P8-T01 | 4 |

| | | | | | |
|---|---|---|---|---|---|
| **P8-T03** | Webhooks System | Outgoing webhooks: event subscription model per tenant (sale confirmed, invoice paid, stock low, lead created), HMAC-SHA256 signed payloads, at-least-once delivery with exponential backoff and dead-letter status, delivery log UI. Check `OCA/server-tools` first.<br>**Acceptance**: a subscribed test endpoint receives signed events, with retries proven by a flaky-receiver test. | `[DEV-1]` | P8-T02 | 4 |
| **P8-T04** | Full Observability Stack | Prometheus + Grafana + `node_exporter` + `postgres_exporter` + a custom Odoo metrics exporter (request latency per tenant DB, worker saturation, cron duration, queue depth, pool saturation). Alert rules: CPU, disk, connection pool, SSL expiry, backup/WAL failure, replication lag. Supersedes the lightweight P2-T10 probes.<br>**Acceptance**: Grafana shows live per-tenant latency; a test alert fires to Discord. | `[DEV-1]` | P2-T10 | 3 |
| **P8-T05** | Audit Trail | Evaluate OCA `auditlog` first. Field-level change tracking on critical models ( `account.move` , `res.partner` , `sale.order` , `res.users` , all ncollection platform models), per-record history view, CSV export, retention policy, tamper evidence (hash chaining if feasible).<br>**Acceptance**: changing an invoice amount produces an audit entry with old/new values, user, IP, and timestamp. | `[DEV-2]` | P1-T07 | 4 |
| **P8-T06** | Developer SDKs | Client SDKs generated from the OpenAPI spec: Python and Node.js packages with auth handling, retries, pagination helpers; published under the NCollection scope; quickstart docs with real examples.<br>**Acceptance**: a 10-line SDK script creates a contact and lists invoices against staging. | `[DEV-2]` | P8-T02 | 5 |
| **P8-T07** | API Documentation Portal | Branded Redoc/Swagger UI from the OpenAPI spec with a try-it-out sandbox against a demo tenant, authentication guide, webhook integration guide, rate-limit documentation.<br>**Acceptance**: an external developer can go from zero to first API call using only the portal. | `[DEV-3]` | P8-T02 | 3 |
| **P8-T08** | Integration Directory UI | Curated integration catalog inside the workspace ( `ncollection.integration.listing` ): categories, search, per-integration setup guides, request-an-integration form. This is the curated precursor to the (deferred) Phase 9 marketplace.<br>**Acceptance**: 5 seeded listings browsable with working setup links. | `[DEV-3]` | P8-T02 | 5 |
| **P8-T09** | Per-Tenant Cost & Usage Dashboard | Two dashboards per ARCHITECTURE_DATA_PLATFORM.md §14: (1) internal ops view (Admin DB only) showing per-tenant infra cost attribution — DB size, filestore size, worker/CPU time share, backup storage share, LLM token spend if Phase 5 shipped — to catch unprofitable tenants before quarter-end, (2) optional tenant-facing usage view against plan limits, built only if a usage-based pricing tier is introduced. Data comes from a scheduled probe ( `pg_database_size()` , filestore `du` , backup-repo size) written to `ncollection.tenant` rollup fields — NEVER computed live on dashboard render.<br>**Reason**: §12's capacity model gives platform-wide cost by tenant-count tier, but nothing today attributes cost to an individual tenant — a gap identified in the July 2026 review.<br>**Acceptance**: the ops dashboard shows accurate per-tenant cost breakdown refreshed at least hourly; the internal view never queries tenant business data, only pre-aggregated metadata in the Admin DB. | `[DEV-2]` | P8-T04, P5-T02 | 3 |

---

## Phase 9: Marketplace (DEFERRED)

**Priority**: P3 — DEFERRED — **executed AFTER Phase 10, and only with proven tenant demand Objective**: Full third-party addon marketplace with publisher ecosystem.

> [!WARNING] **Deferral rationale (v5.0)**: Building a third-party developer ecosystem before the core platform is enterprise-grade is a distraction with negative expected value — every marketplace feature multiplies the security surface (arbitrary third-party code installed into tenant DBs). The curated Integration Directory (P8-T08) covers the near-term need. Re-evaluate after Phase 10 with real tenant demand data.

| ID | Task Name | Description | Assigned | Dependencies | Est. Days |
|---|---|---|---|---|---|
| **P9-T01** | Marketplace Backend Models | `ncollection.marketplace.app` with versioning, Odoo-version compatibility matrix, pricing models (free/one-time/recurring), publisher accounts, review states. **Acceptance**: an app record moves draft→submitted→approved→published with correct gating. | `[DEV-1]` | P8-T02 | 4 |
| **P9-T02** | App Submission & Compatibility Pipeline | Publisher upload (zip), automated lint + install test against a disposable DB in CI, security scan (no raw SQL/eval, manifest policy), signing of approved packages. **Acceptance**: a malformed addon is auto-rejected with a readable report; a clean addon passes to review. | `[DEV-1]` | P9-T01 | 4 |
| **P9-T03** | App Installation Engine | One-click install of a signed marketplace app into a tenant DB — sandbox install test, dependency resolution, rollback on failure, per-tenant installed-apps registry, update notifications. **Acceptance**: install, upgrade, and rollback of a sample app on a staging tenant. | `[DEV-1]` | P9-T02 | 5 |
| **P9-T04** | Developer Portal | Publisher self-service: submission wizard, docs requirements, review status tracking, revenue-share configuration, payout statements, install/rating analytics. **Acceptance**: an external publisher account can submit and track an app without admin help. | `[DEV-2]` | P9-T02 | 5 |
| **P9-T05** | Review & Rating System | Verified-install-only reviews, moderation queue, weighted average ratings, featured/trending computation, abuse reporting. **Acceptance**: only tenants with the app installed can review; moderation hides flagged content. | `[DEV-2]` | P9-T01 | 3 |
| **P9-T06** | Marketplace Storefront UI | Public branded storefront: category browse, search, app detail pages with screenshots and reviews, SEO metadata, bilingual. **Acceptance**: Lighthouse SEO score above 90; the storefront lists published apps live from the backend. | `[DEV-3]` | P9-T01 | 5 |
| **P9-T07** | In-Workspace Marketplace Widget | OWL component to browse/install from within the ERP, entitlement checks against the subscription plan, installed-apps management screen. **Acceptance**: one-click install from inside a staging tenant workspace. | `[DEV-3]` | P9-T03 | 4 |

---

## Phase 10: Enterprise Readiness

**Priority**: P3 — executed BEFORE Phase 9 **Objective**: Harden the platform for enterprise-scale operations: high availability, horizontal scaling, advanced security, multi-region data residency, and compliance.

| ID | Task Name | Description | Assigned | Dependencies | Est. Days |
|---|---|---|---|---|---|
| **P10-T01** | HA Foundation: PostgreSQL Replication | PostgreSQL streaming replication (primary + hot standby) with pgBackRest integration, documented promotion procedure, replication monitoring (lag alerts via P8-T04). See ARCHITECTURE_DATA_PLATFORM.md §6. **Acceptance**: a controlled failover drill completes with under 60s data-plane interruption and zero data loss. | `[DEV-1]` | P8-T04 | 4 |
| **P10-T02** | Automated Failover & Zero-Downtime Deploys | Patroni (or repmgr — evaluate) for automatic promotion, HAProxy/PgBouncer re-pointing, blue-green Odoo deployment with health-gated switchover, session draining. **Acceptance**: killing the primary DB self-heals without operator action; a deploy causes zero dropped requests in a load test. | `[DEV-1]` | P10-T01 | 4 |
| **P10-T03** | Horizontal Scaling & Tenant Sharding | Multi-node Odoo behind a load balancer, shared session strategy, S3-backed or NFS shared filestore, tenant-to-cluster mapping in the registry enabling a second PostgreSQL cluster (sharding by tenant), capacity runbook. See ARCHITECTURE_DATA_PLATFORM.md §6. **Acceptance**: tenants split across two DB clusters transparently; a load test at 5× baseline passes. | `[DEV-1]` | P10-T02 | 5 |

| | | | | | |
|---|---|---|---|---|---|
| **P10-T04** | Advanced Security & External Pen Test | SSO via external IdP (Keycloak/Auth0) for enterprise tenants, TOTP 2FA for all plans (check `OCA/server-auth`), per-tenant IP allowlists, secrets migration to a vault, third-party penetration test with remediation, SOC 2 readiness gap analysis. **Acceptance**: pen-test report criticals resolved; 2FA enforceable per tenant policy. | `[DEV-1]` | P3-T12 | 5 |
| **P10-T05** | Multi-Region Support | Region-aware tenant placement (UAE data residency), per-region DB clusters + filestores, CDN for static assets, geo-DNS routing, region-scoped backups. **Acceptance**: a tenant provisioned with region=UAE has all data (DB, filestore, backups) physically in the UAE region. | `[DEV-1]` | P10-T03 | 5 |
| **P10-T06** | Enterprise Accounting | Multi-company consolidation within a tenant, intercompany transaction flows, advanced bank reconciliation (evaluate `OCA/account-reconcile`), budget management. **Acceptance**: a two-company tenant produces consolidated statements that reconcile. | `[DEV-2]` | P3-T05 | 5 |
| **P10-T07** | Compliance & Data Governance | UAE PDPL (Federal Decree-Law 45/2021) alignment: consent registry, tenant data export (portability), verified erasure workflow, retention policies, UAE e-invoicing readiness tracking, records-of-processing documentation. See ARCHITECTURE_SECURITY.md §9. **Acceptance**: a tenant offboarding produces a complete export and a certified deletion log. | `[DEV-2]` | P8-T05 | 4 |
| **P10-T08** | Enterprise Onboarding Wizard | Guided setup with industry templates (trading, services, manufacturing), P3-T11 import integration, progress checklist, sample-data toggle. **Acceptance**: a new enterprise tenant reaches a configured, data-loaded workspace in under one day without engineering help. | `[DEV-3]` | P2-T02, P3-T11 | 5 |
| **P10-T09** | White-Label Reseller System | Partner accounts reselling under their own brand: cascading branding (partner brand overrides NCollection defaults), partner dashboard with sub-tenant management, revenue-share reporting, partner-scoped provisioning quotas. **Acceptance**: a partner provisions a sub-tenant carrying partner branding end-to-end. | `[DEV-3]` | P1-T16 | 5 |

---

# 19. Cross-Cutting Concerns

## 19.1 Testing Strategy

| Level | Scope | Tools | Responsibility |
|---|---|---|---|
| **Unit Tests** | Model methods, computed fields, constraints, state machines | `odoo.tests.common.TransactionCase` | Each DEV for own code |
| **Integration Tests** | Cross-model workflows, provisioning pipeline | `odoo.tests.common.HttpCase` | DEV-1 leads |
| **E2E Tests** | Login, routing, isolation, visibility, checkout, lifecycle | **Playwright (P1-T20)** — runs in CI on every PR | DEV-3 owns framework; all DEVs add journeys |
| **Security Tests** | Cross-tenant access, license bypass, role enforcement | Automated (in E2E) + P3-T12 assessment | DEV-1 + DEV-2 |
| **Load Tests** | Multi-tenant performance, concurrent users | k6 / Locust (P3-T03) | DEV-1 |
| **Regression** | Cumulative checklist re-run at every phase gate | E2E suite + checklist | Gate owner |

## 19.2 Documentation Requirements

Every completed phase must produce:

1. **Technical Docs**: model schemas, API specs, configuration guides
2. **User Guides**: tenant admin and end-user documentation
3. **Admin Guides**: how NCollection staff operate the platform

4. **Runbooks**: `RUNBOOK_SECURITY.md`, `RUNBOOK_INCIDENTS.md`, backup/restore, scaling, deploy/rollback

### 19.3 Environment Progression

```
Local (docker-compose.dev.yml)
    → Staging (Hetzner VPS #1 — auto-deploy on merge to develop, P2-T07)
        → Production (Hetzner VPS #2 — manual promotion from main, P3-T13)
```

### 19.4 Definition of Ready / Definition of Done

**A task is READY when**: dependencies are Done, the OCA check is recorded on the issue, acceptance criteria are understood, and the assignee has estimated it fits the sprint.

**A task is DONE when**: CI green (lint + tests + scans), unit tests added, E2E journeys updated where relevant, PR approved by 1 reviewer, merged to `develop`, docs updated, and the acceptance criteria on the issue are checked off with evidence.

---

> **Document End** This is a living document. Update after each sprint to reflect completed tasks, new decisions, and architectural changes. Never redesign completed milestones unless explicitly requested. Task tables in this file are machine-readable by `scripts/github_issue_sync.py` — keep the 6-column row format intact.